



CyberBoxSecur

CiberBoxSecurBD

Projeto prático da Unidade Curricular de Bases de Dados

Curso: Tecnologias e Design de Multimédia

Ano letivo: 2025/2026

Estudante: Daniel Teixeira

Número: 27645

Docentes: Paulo Tomé e Mauro Lima

Julho de 2026

Declaração de âmbito

Âmbito académico. Este documento acompanha uma aplicação Django para a UC de Bases de Dados. As entidades do domínio são persistidas exclusivamente por SQL direto através do ficheiro `ciberbox/basededados.py`; não existe `models.py` com modelos do domínio nem são utilizados QuerySets ou ModelForms.

O tema CiberBoxSecur foi adotado a partir do trabalho de Projeto Integrado III. Esta escolha é autorizada pelo enunciado de Bases de Dados, que permite usar o mesmo tema da UC de PI III (enunciado_base de dados.pdf, p. 3).

Os entregáveis considerados são os expressos na página 4 do enunciado: modelo conceptual, modelo lógico, modelo físico, script de criação e inicialização, aplicação Django com SQL e não ORM e dashboard da Ficha 9.

A importação de ativos e incidentes a partir de Excel foi implementada como funcionalidade adicional solicitada pelo estudante. A aplicação usa os dois ficheiros Excel fornecidos como referência, mas também inclui modelos simplificados próprios para demonstração.

Resumo

O CiberBoxSecurBD é um sistema de apoio a uma empresa de cibersegurança. Permite gerir organizações clientes, utilizadores, contactos, ativos tecnológicos, incidentes, documentos, avaliações de risco, pedidos de suporte, mensagens, histórico de estados, importações Excel e logs de atividade. O modelo foi concebido para preservar integridade, histórico e rastreabilidade, ao mesmo tempo que suporta as cinco consultas agregadas obrigatórias do dashboard.

A solução utiliza PostgreSQL, Django e Psycopg. A camada de apresentação e controlo é Django, enquanto a persistência fica isolada em funções de SQL parametrizado. Foram implementadas transações, constraints, índices, autorização por perfil e organização, hash de passwords, armazenamento privado de documentos e importação Excel com pré-visualização e relatório por linha.

Índice de conteúdos

1. Enquadramento e requisitos
 2. Análise do domínio
 3. Modelo conceptual
 4. Modelo lógico e normalização
 5. Modelo físico PostgreSQL
 6. Scripts SQL e dados de demonstração
 7. Dashboard obrigatório
 8. Aplicação Django sem ORM
 9. Importação de Excel
 10. Segurança e integridade
 11. Testes e validação
 12. Instalação e demonstração
 13. Limitações e trabalho futuro
 14. Conclusão
- Referências e anexos

1. Enquadramento e requisitos

1.1. Requisitos formais da UC

O enunciado do trabalho prático, ano letivo 2025/2026, define que o tema pode ser igual ao trabalho de PI III. Na página 4, exige um ZIP com os três níveis de modelação, script de criação e inicialização e código Django segundo as Fichas 8 e 9. A persistência deve usar SQL e não ORM. A Ficha 7 acrescenta que deve existir um ficheiro basededados.py com funções CRUD que executam queries SQL e que as views devem utilizar essas funções. A Ficha 8 aplica os passos da Ficha 7 ao tema do projeto.

Origem	Página	Exigência	Evidência na entrega
Enunciado BD	3	Tema igual ao de PI III	Domínio CiberBoxSecur
Enunciado BD	4	Modelos conceptual, lógico e físico	Pasta modelos/ e relatório
Enunciado BD	4	Script de criação e inicialização	Pasta sql/
Enunciado BD	4	Django com SQL, sem ORM	ciberbox/basededados.py
Ficha 7	1-2	Configuração, formulários, CRUD SQL e views	Aplicação Django
Ficha 8	1	Aplicar Ficha 7 ao projeto prático	Todos os módulos do domínio
Ficha 9	1	Cinco indicadores por SELECT no basededados.py	Dashboard e sql/04_queries_dashboard.sql

1.2. Entregáveis produzidos

- Modelo conceptual explicado e diagrama IDEF1X simplificado em PNG, SVG e DOT.
- Modelo lógico relacional, chaves, atributos obrigatórios/opcionais e normalização até 3FN.
- Modelo físico PostgreSQL com tipos, constraints, índices e regras referenciais.
- Scripts separados para limpeza opcional, criação, inicialização, dados de demonstração, dashboard e validação.
- Aplicação Django com CRUD e acesso SQL direto.
- Dashboard com os cinco indicadores da Ficha 9.
- Importação Excel de ativos e incidentes, com validação, pré-visualização e relatório.
- README, verificador automático, modelos Excel e preparação para defesa.

2. Análise do domínio

2.1. Problema de negócio

A CiberBoxSecur acompanha organizações na gestão de informação de cibersegurança. Para cada organização é necessário manter contactos relevantes, inventário tecnológico, incidentes, documentação e avaliações de risco/conformidade. A relação entre cliente e equipa é complementada por pedidos de suporte com mensagens e histórico de estados.

2.2. Atores e permissões

Perfil	Acesso principal	Restrições
Administrador	Todos os clientes; utilizadores; logs; CRUD global; dashboard.	Não pode desativar a própria conta pela interface.
Colaborador	Todos os clientes; dados de segurança; pedidos; importações; dashboard.	Não gere utilizadores nem consulta logs globais.
Cliente	Apenas organizações às quais está associado; regista e consulta os seus dados.	Não vê clientes alheios, utilizadores globais ou logs.

2.3. Regras de negócio

1. Cada utilizador possui exatamente um perfil e um email único.
2. Cada organização cliente possui um NIF único de nove dígitos.
3. Um utilizador Cliente só acede a organizações presentes em utilizadores_clientes.
4. O estado de conformidade exibido corresponde à avaliação de risco mais recente.
5. O número de inventário e o código de incidente são únicos dentro de cada cliente.
6. Um pedido conserva estado atual para consulta eficiente e histórico das transições para auditoria.
7. Os documentos são privados; a base de dados guarda metadados e caminho, não o conteúdo binário.
8. Uma importação Excel só é persistida depois de uma pré-visualização e confirmação explícita.
9. Uma linha inválida do Excel é rejeitada sem anular as restantes linhas válidas do lote.
10. As ações relevantes geram logs sem guardar passwords nem conteúdo dos documentos.

2.4. Rastreabilidade entre requisitos e dados

Requisito	Entidades principais	Operação/consulta
Perfis e utilizadores	perfis, utilizadores, utilizadores_clientes	Login, autorização e dashboard por perfil
Clientes e contactos	clientes, contactos_clientes	CRUD e ficha do cliente
Ativos	ativos_tecnologicos, importacoes_excel	CRUD e importação Excel
Incidentes	incidentes, importacoes_excel	CRUD, importação e top 5
Documentos	documentos	Upload privado e total por mês
Risco/NIS2	avaliacoes_risco, estados_conformidade	Histórico e clientes por estado
Pedidos	pedidos, estados_pedidos, mensagens_pedidos, historico_estados_pedidos	Fluxo e tempo médio
Auditoria	logs_atividade, linhas_importacao	Rastreabilidade

3. Modelo conceptual

3.1. Critérios de modelação

O modelo conceptual representa entidades, atributos essenciais e cardinalidades sem depender dos tipos físicos. Foram usadas entidades no plural e uma notação IDEF1X simplificada: as entidades independentes possuem identidade própria; as dependentes requerem a existência de uma entidade pai; utilizadores_clientes é associativa e identificada pelas chaves dos dois pais.

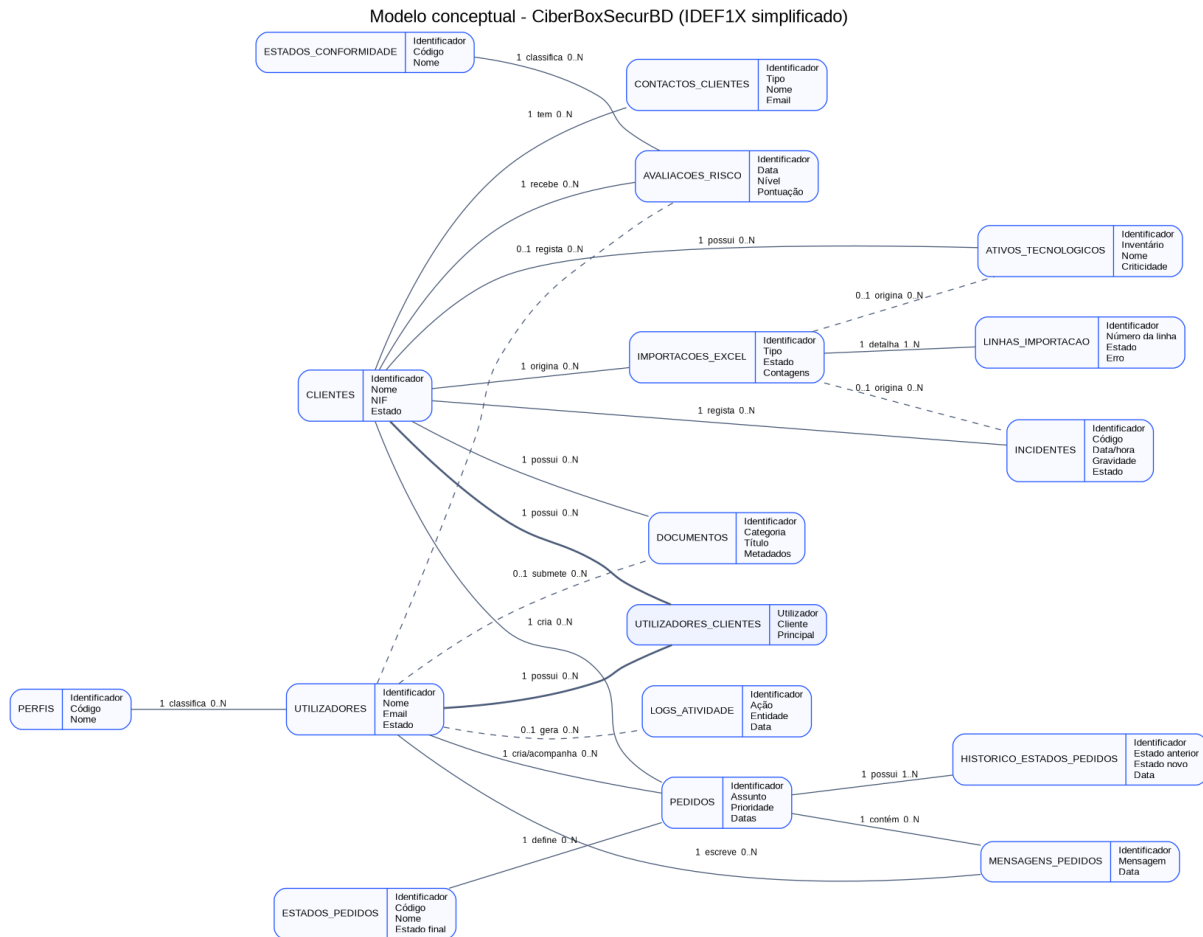


Figura 1 - Modelo conceptual do CiberBoxSecurBD (IDEF1X simplificado).

3.2. Entidades e justificação

Entidade	Tipo	Justificação
PERFIS	Independente	Vocabulário dos perfis exigidos.
UTILIZADORES	Independente	Identidade, autenticação e autoria.
CLIENTES	Independente	Organizações acompanhadas.
UTILIZADORES_CLIENTES	Associativa dependente	Resolve N:M entre contas Cliente e organizações.
CONTACTOS_CLIENTES	Dependente	Responsáveis de segurança e contactos permanentes.
ESTADOS_CONFORMIDADE	Independente	Vocabulário dos estados NIS2.
AVALIACOES_RISCO	Dependente	Histórico de risco e conformidade.
ATIVOS_TECNOLOGICOS	Dependente	Inventário tecnológico.
INCIDENTES	Dependente	Ocorrências de segurança.
DOCUMENTOS	Dependente	Metadados de relatórios, pentests e evidências.
ESTADOS_PEDIDOS	Independente	Ciclo de vida controlado.
PEDIDOS	Dependente	Tickets e pedidos dos clientes.
MENSAGENS_PEDIDOS	Dependente	Interação no pedido.
HISTORICO_ESTADOS_PEDIDOS	Dependente	Auditoria das transições.
IMPORTACOES_EXCEL	Dependente	Cabeçalho e contagens do lote.
LINHAS_IMPORTACAO	Dependente	Resultado individual de cada linha.
LOGS_ATIVIDADE	Dependente opcional	Auditoria do utilizador ou sistema.

3.3. Cardinalidades essenciais

- PERFIS 1:N UTILIZADORES: cada utilizador tem um perfil; um perfil pode classificar vários utilizadores.

- UTILIZADORES N:M CLIENTES: relação resolvida por UTILIZADORES_CLIENTES.
- CLIENTES 1:N para contactos, avaliações, ativos, incidentes, documentos, pedidos e importações.
- ESTADOS_CONFORMIDADE 1:N AVALIACOES_RISCO.
- IMPORTACOES_EXCEL 1:N LINHAS_IMPORTACAO e 0:N ATIVOS/INCIDENTES; a origem é opcional porque há criação manual.
- ESTADOS_PEDIDOS 1:N PEDIDOS e 1:N HISTORICO_ESTADOS_PEDIDOS.
- PEDIDOS 1:N MENSAGENS_PEDIDOS e 1:N HISTORICO_ESTADOS_PEDIDOS.
- UTILIZADORES 1:N sobre registos que conservam autoria; algumas participações são opcionais para preservar histórico.

4. Modelo lógico e normalização

4.1. Transformação relacional

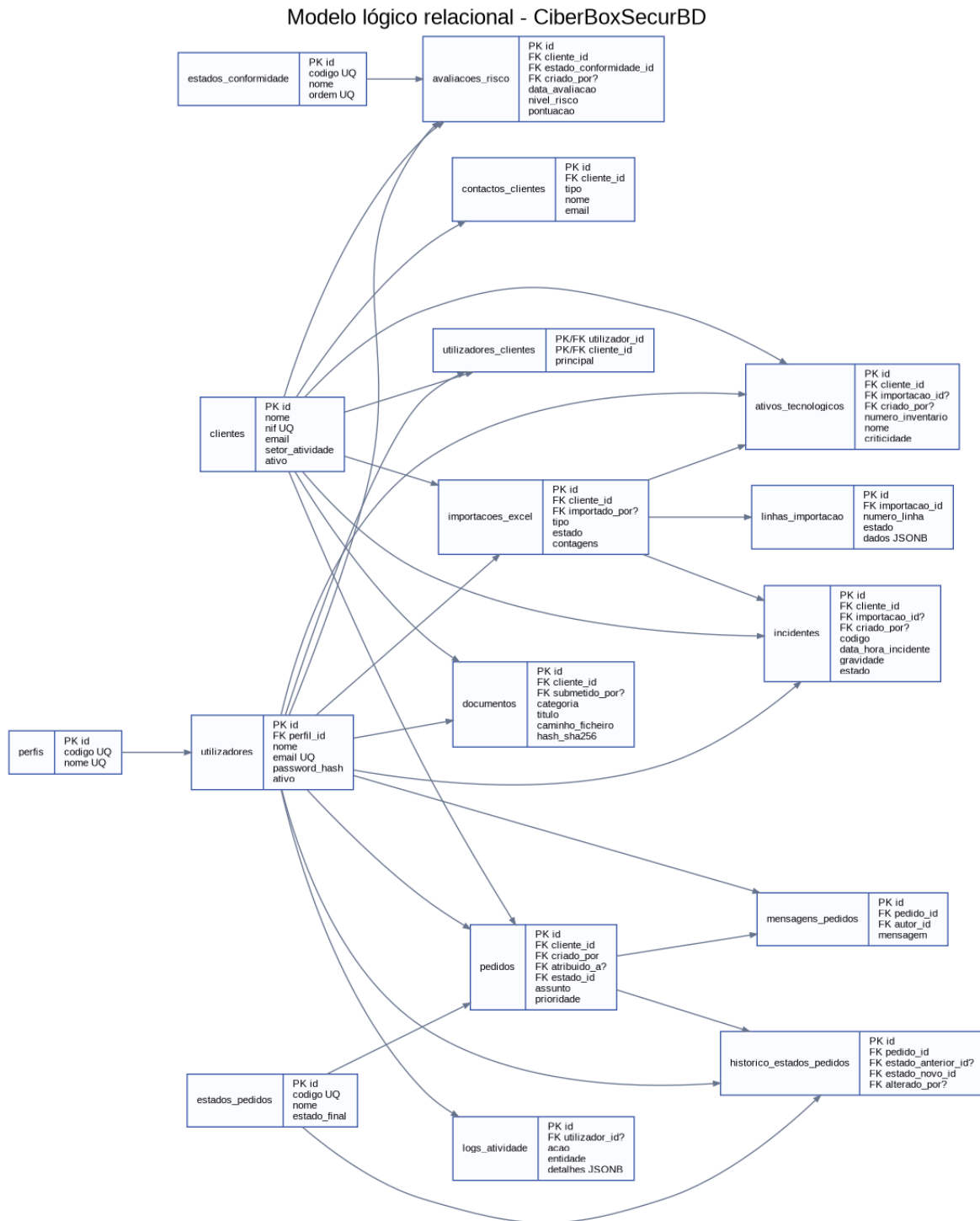


Figura 2 - Modelo lógico relacional com PK, FK e chaves únicas principais.

4.2. Relações e chaves

Relação	PK	Chaves candidatas/UNIQUE	FK principais
perfis	id	codigo; nome	-
utilizadores	id	email	perfil_id
clientes	id	nif	-
utilizadores_clientes	utilizador_id + cliente_id	-	utilizador_id; cliente_id
contactos_clientes	id	cliente_id + tipo + email	cliente_id
estados_conformidade	id	codigo; nome; ordem	-
avaliacoes_risco	id	cliente_id + data_avaliacao	cliente_id; estado_conformidade_id; criado_por
importacoes_excel	id	-	cliente_id; importado_por
linhas_importacao	id	importacao_id + numero_linha	importacao_id
ativos_tecnologicos	id	cliente_id + numero_inventario	cliente_id; importacao_id; criado_por
incidentes	id	cliente_id + codigo	cliente_id; importacao_id; criado_por
documentos	id	nome_ficheiro_guardado	cliente_id; submetido_por
estados_pedidos	id	codigo; nome; ordem	-
pedidos	id	-	cliente_id; criado_por; atribuido_a; estado_id
mensagens_pedidos	id	-	pedido_id; autor_id
historico_estados_pedidos	id	-	pedido_id; estados; alterado_por
logs_atividade	id	-	utilizador_id

4.3. Atributos multivalor e relações N:M

Os contactos, ativos, incidentes, documentos, mensagens e alterações de estado seriam atributos multivalor se fossem guardados dentro do cliente ou pedido. Foram convertidos em relações próprias 1:N. A associação de utilizadores a clientes é N:M e foi convertida em utilizadores_clientes, com PK composta. Esta transformação elimina listas dentro de colunas e permite integridade referencial.

4.4. Demonstração de normalização

Forma normal	Verificação	Exemplo no projeto
1FN	Valores atômicos e ausência de grupos repetidos.	Contactos, mensagens e ativos possuem tabelas próprias.
2FN	Atributos não-chave dependem da chave completa.	Em utilizadores_clientes, principal depende do par utilizador/cliente.
3FN	Não há dependências transitivas entre atributos não-chave.	O nome do perfil/estado/cliente é obtido por FK e não repetido.

Os totais do dashboard não são armazenados: são derivados por COUNT, AVG e agrupamentos. O JSONB em logs_atividade.detalhes e linhas_importacao.dados é usado apenas para auditoria variável; não substitui os atributos relacionais necessários às operações e consultas.

5. Modelo físico PostgreSQL

5.1. Convenções e tipos

Elemento	Decisão	Justificação
Nomes	Português, plural e snake_case	Consistência com o domínio e legibilidade.
PK	BIGSERIAL; SMALLSERIAL em vocabulários	Chaves artificiais estáveis.
Datas	TIMESTAMPTZ	Instantes consistentes com fuso horário.
Email	CITEXT	Unicidade sem diferenciar maiúsculas/minúsculas.
Rede	INET e MACADDR	Validação nativa de endereços.
Montantes	NUMERIC	Evita erro de arredondamento de FLOAT.
Auditoria variável	JSONB	Detalhes de logs e reprodução das linhas Excel.

Elemento	Decisão	Justificação
Texto livre	TEXT	Descrições sem limite artificial inadequado.

5.2. Integridade de entidade e referencial

- PRIMARY KEY impede linhas sem identidade e valores duplicados.
- FOREIGN KEY impede referências a perfis, clientes, estados ou utilizadores inexistentes.
- NOT NULL representa participação obrigatória.
- UNIQUE concretiza email, NIF, códigos e chaves candidatas compostas.
- CHECK limita NIF, perfis, criticidade, estados, pontuações, contagens e coerência temporal.
- ON DELETE RESTRICT protege clientes e vocabulários com histórico.
- ON DELETE CASCADE é reservado a entidades totalmente dependentes, como linhas de importação e mensagens.
- ON DELETE SET NULL preserva o registo quando o autor deixa de existir.

5.3. Constraints exemplificativas

```

CONSTRAINT ck_clientes_nif CHECK (nif ~ '^[0-9]{9}$')
CONSTRAINT uq_ativos_cliente_inventario UNIQUE (cliente_id, numero_inventario)
CONSTRAINT uq_incidentes_cliente_codigo UNIQUE (cliente_id, codigo)
CONSTRAINT ck_avaliacoes_pontuacao CHECK (pontuacao BETWEEN 0 AND 100)
CONSTRAINT ck_pedidos_datas CHECK (
  resolvido_em IS NULL OR resolvido_em >= criado_em
)

```

5.4. Índices

Os índices foram definidos a partir de filtros e JOINS reais: cliente e data em avaliações, incidentes e documentos; estado e data em pedidos; pedido em mensagens e histórico; utilizador/data e entidade em logs. A unicidade já cria índices para email, NIF e chaves candidatas.

Índice/colunas	Consulta suportada
avaliacoes_risco(cliente_id, data_avaliacao DESC, id DESC)	Última avaliação e estado atual.
incidentes(cliente_id, data_hora_incidente DESC)	Listagem e contagem de incidentes.
documentos(cliente_id, submetido_em DESC)	Documentos por cliente e mês.
pedidos(estado_id, criado_em)	Estado dos tickets.
mensagens_pedidos(pedido_id, criado_em)	Conversação cronológica.
logs_atividade(utilizador_id, criado_em DESC)	Auditoria por utilizador.

6. Scripts SQL e dados de demonstração

6.1. Ordem de execução

1. 00_limpeza_opcional.sql - remove objetos do projeto de forma controlada; só deve ser usado quando se pretende reinicializar.
2. 01_criacao.sql - ativa CTEXT, cria tabelas, constraints e índices por dependência.
3. 02_inicializacao.sql - insere perfis, estados de conformidade e estados de pedidos de forma idempotente.
4. 03_dados_demonstracao.sql - cria seis clientes, quatro utilizadores e dados suficientes para as cinco consultas.
5. 04_queries_dashboard.sql - contém as SELECT pedidas na Ficha 9.
6. 05_queries_validacao.sql - confirma volumes, órfãos, duplicados e coerência.

6.2. Repetibilidade

A criação usa CREATE TABLE IF NOT EXISTS e os vocabulários usam INSERT ... ON CONFLICT. Para uma repetição integral e previsível recomenda-se executar o inicializador com --limpar, que aplica primeiro o script de limpeza. Não se usa sequelize.sync, migrations de ORM nem criação automática de modelos.

```
python scripts/inicializar_bd.py --limpar
python manage.py check
python scripts/verificar_projeto.py
python manage.py runserver
```

6.3. Dados de demonstração

Os dados foram desenhados para que cada consulta produza resultados demonstráveis: seis clientes repartidos igualmente pelos três estados de conformidade, incidentes com contagens diferentes, documentos em vários meses, utilizadores nos três perfis e pedidos em todos os estados. As passwords de demonstração são convertidas em hash PBKDF2-SHA256 antes de serem inseridas; o texto simples não é guardado.

7. Dashboard obrigatório da Ficha 9

A Ficha 9 exige que os cinco indicadores sejam obtidos por SELECT e executados por funções adicionadas a basedados.py. Cada função abre um cursor com context manager, executa SQL e transforma as colunas em dicionários. A view dashboard chama obter_dashboard(), que agrega os cinco resultados e os envia para o template.

7.1. Clientes por estado de conformidade NIS2

Função Django: dashboard_clientes_por_conformidade(). Tabelas: estados_conformidade, avaliacaoes_risco.

```
- numero de clientes por estado de conformidade NIS2.
WITH ultima_avaliacao AS (
  SELECT DISTINCT ON (cliente_id)
    cliente_id, estado_conformidade_id
  FROM avaliacaoes_risco
  ORDER BY cliente_id, data_avaliacao DESC, id DESC
)
SELECT ec.codigo, ec.nome AS estado, COUNT(ua.cliente_id)::INTEGER AS numero_clientes
FROM estados_conformidade ec
LEFT JOIN ultima_avaliacao ua ON ua.estado_conformidade_id = ec.id
GROUP BY ec.id, ec.codigo, ec.nome, ec.ordem
ORDER BY ec.ordem;
```

A CTE seleciona a avaliação mais recente por cliente com DISTINCT ON. O LEFT JOIN mantém estados com zero clientes. COUNT agrega por estado.

Estado	Número de clientes
Conforme	2
Em avaliacao	2
Com pendencias	2

7.2. Top 5 clientes com mais incidentes

Função Django: dashboard_top_clientes_incidentes(). Tabelas: clientes, incidentes.

```
- top 5 clientes com mais incidentes de segurança.
SELECT c.id, c.nome, COUNT(i.id)::INTEGER AS total_incidentes
FROM clientes c
JOIN incidentes i ON i.cliente_id = c.id
GROUP BY c.id, c.nome
ORDER BY total_incidentes DESC, c.nome
LIMIT 5;
```

O INNER JOIN exclui clientes sem incidentes. GROUP BY agrega por cliente; ORDER BY decrescente e LIMIT 5 produzem o ranking.

Cliente	Total de incidentes
Zeta Industria, S.A.	5
Gamma Transportes, S.A.	4
Beta Energia, Lda.	3
Alpha Saude, S.A.	2
Epsilon Municipal, E.M.	2

7.3. Documentos submetidos por cliente e mês

Função Django: dashboard_documentos_por_cliente_mes(). Tabelas: clientes, documentos.

```
- total de documentos submetidos por cliente e por mes.
SELECT c.id,
       c.nome,
       DATE_TRUNC('month', d.submetido_em)::DATE AS mes,
       COUNT(d.id)::INTEGER AS total_documentos
FROM clientes c
JOIN documentos d ON d.cliente_id = c.id
GROUP BY c.id, c.nome, DATE_TRUNC('month', d.submetido_em)::DATE
ORDER BY mes DESC, c.nome;
```

DATE_TRUNC normaliza cada data para o primeiro dia do mês. O agrupamento combina cliente e mês; COUNT calcula documentos.

Cliente	Mês	Total
Zeta Industria, S.A.	2026-06-01	2
Epsilon Municipal, E.M.	2026-05-01	1
Delta Tecnologia, Lda.	2026-04-01	1
Beta Energia, Lda.	2026-03-01	1
Gamma Transportes, S.A.	2026-03-01	2
Alpha Saude, S.A.	2026-02-01	1
Beta Energia, Lda.	2026-02-01	1
Alpha Saude, S.A.	2026-01-01	2

7.4. Distribuição de utilizadores por perfil

Função Django: dashboard_utilizadores_por_perfil(). Tabelas: perfis, utilizadores.

```
- distribuicao de utilizadores por perfil.
SELECT p.codigo,
       p.nome AS perfil,
       COUNT(u.id)::INTEGER AS total_utilizadores,
       COUNT(u.id) FILTER (WHERE u.ativo)::INTEGER AS utilizadores_ativos
FROM perfis p
LEFT JOIN utilizadores u ON u.perfil_id = p.id
GROUP BY p.id, p.codigo, p.nome
ORDER BY p.id;
```

O LEFT JOIN preserva perfis sem utilizadores. COUNT calcula total e FILTER calcula apenas contas ativas.

Perfil	Total	Ativos
Administrador	1	1
Colaborador	1	1
Cliente	2	2

7.5. Estado dos pedidos e tempo médio de resolução

Função Django: dashboard_estado_pedidos_tempo_medio(). Tabelas: estados_pedidos, pedidos.

```
- estado dos pedidos e tempo medio de resolucao em horas.
SELECT ep.codigo,
       ep.nome AS estado,
       COUNT(p.id)::INTEGER AS total_pedidos,
       ROUND(
         AVG(EXTRACT(EPOCH FROM (p.resolvido_em - p.criado_em))) / 3600.0)
       FILTER (WHERE p.resolvido_em IS NOT NULL),
       2
) AS tempo_medio_resolucao_horas
```

```
FROM estados_pedidos ep
LEFT JOIN pedidos p ON p.estado_id = ep.id
GROUP BY ep.id, ep.codigo, ep.nome, ep.ordem
ORDER BY ep.ordem;
```

EXTRACT(EPOCH) converte o intervalo resolvido_em - criado_em em segundos; a divisão por 3600 devolve horas. AVG é filtrado para pedidos resolvidos.

Estado	Total	Média (horas)
Aberto	1	-
Em análise	1	-
Aguarda cliente	1	-
Resolvido	2	57.00
Fechado	1	48.00

7.6. Casos sem dados

Nas consultas de conformidade, perfis e pedidos foi usado LEFT JOIN para que as categorias continuem visíveis com contagem zero. O tempo médio é NULL quando nenhum pedido daquele estado possui resolvido_em; o template apresenta um hífen. O top 5 e os documentos por mês não mostram linhas quando não existem factos associados, o que é semanticamente adequado.

8. Aplicação Django sem ORM

8.1. Arquitetura

Camada	Ficheiros	Responsabilidade
Configuração	ciberbox_bd/settings.py, urls.py	PostgreSQL, cookies, segurança, rotas.
Apresentação	templates/, static/	HTML, formulários, tabelas e dashboard.
Controlo	ciberbox/views.py, decorators.py	Fluxos HTTP, autorização e mensagens.
Validação	ciberbox/forms.py	Formulários Django comuns, sem ModelForm.
Persistência	ciberbox/basededados.py	CRUD e SELECT com SQL parametrizado.
Excel	ciberbox/excel_import.py	Deteção, mapeamento, validação e pré-visualização.
Ficheiros	ciberbox/storage.py	Nomes aleatórios, hash, caminhos privados.

8.2. Fluxo de uma operação CRUD

1. O URL encaminha o pedido para uma view.
2. O decorator confirma sessão, perfil e, quando aplicável, organização.
3. Um Form valida e normaliza os dados recebidos.
4. A view chama uma função específica de basededados.py.
5. A função usa connection.cursor() como context manager e placeholders %s.
6. O PostgreSQL verifica constraints e executa a transação.
7. A ação relevante é registada em logs_atividade.
8. A view apresenta sucesso ou uma mensagem genérica de erro; o detalhe técnico fica no log do servidor.

8.3. Exemplo de SQL parametrizado

```
def obter_cliente(cliente_id):
    with connection.cursor() as cursor:
        cursor.execute(
            "SELECT * FROM clientes WHERE id=%s",
            [cliente_id],
```

```
)
return _dictfetchone(cursor)
```

O valor do utilizador não é concatenado com o SQL. O driver envia a instrução e os parâmetros separadamente, evitando que caracteres do valor alterem a estrutura da query. As únicas f-strings usadas na camada de dados montam fragmentos fixos escolhidos pelo programa, nunca texto arbitrário do utilizador.

8.4. Transações e rollback

Operações compostas, como criação de pedidos com histórico e importação Excel, usam `transaction.atomic()`. Na importação existe ainda um savepoint por linha: uma violação de UNIQUE ou um IP inválido rejeita essa linha, regista o erro e permite continuar o lote. Uma falha fora do savepoint faz rollback da operação completa.

8.5. Ausência de ORM

- Não existe `ciberbox/models.py` com entidades do domínio.
- Não existem classes herdadas de `django.db.models.Model`.
- Não existem `QuerySet`, `objects.filter()`, `objects.create()` ou `ModelForm`.
- O Django é usado para HTTP, templates, formulários, sessões e ligação configurada ao PostgreSQL.
- A persistência de negócio é executada no ficheiro `basededados.py`.

9. Importação de Excel

9.1. Ficheiros analisados

Ficheiro	Folhas relevantes	Informação
Exemplo inventario p_CNCS.xlsx	Ponto_de_Contacto_Permanente; Resp__Segurança; Lista de Ativos	Contactos CNCS, responsáveis e inventário com serviço, software/equipamento, versão, IP, FQDN, fabricante e risco.
Exemplo Lista Activos _incidentes.xlsx	ASSETS; Incidents; Apoio	Inventário detalhado e incidentes com datas, descrição, impacto, resposta, gravidade e encerramento.

9.2. Mapeamento de ativos

Coluna do Excel	Campo físico	Regra
N.º inventário	numero_inventario	Opcional, mas único por cliente quando preenchido.
Tipo de equipamento	tipo_equipamento	Opcional.
Nome	nome	Obrigatório.
Modelo/versão	modelo-versao	Opcional.
IP	endereco_ip	Validado como endereço IP.
Endereço HW/MAC	endereco_mac	Validado no formato MAC.
Criticidade/classificação	criticidade	Normalizada para RESIDUAL, BAIXA, MEDIA, ALTA ou CRITICA.
Responsável/contacto	responsavel_nome/contacto	Opcional.
Comunicado CNCS / programa de risco	booleanos	Sim/S/True/1 convertidos para verdadeiro.

9.3. Mapeamento de incidentes

Coluna do Excel	Campo físico	Regra
ID/código	codigo	Obrigatório e único dentro do cliente.
Data e hora	data_hora_incidente	Obrigatória; combina colunas separadas.

Coluna do Excel	Campo físico	Regra
Tipo	tipo_incidente	Obrigatório.
Descrição	descricao	Obrigatória.
Utilizadores afetados	utilizadores_afetados	Inteiro >= 0.
Dados comprometidos	dados_comprometidos	Booleano.
IP atacante	ip_atacante	Validado quando existe.
Gravidade	gravidade	Normalizada para cinco níveis.
Estado/encerramento	estado, encerrado_em	Coerência validada por constraints.

9.4. Fluxo seguro

1. Aceitar apenas .xlsx até 5 MB.
2. Guardar uma cópia com nome físico aleatório fora da pasta estática.
3. Selecionar folha por nomes conhecidos e detetar o cabeçalho nas primeiras linhas.
4. Normalizar acentos, maiúsculas e variantes dos títulos.
5. Validar cada linha sem escrever na base de dados.
6. Mostrar pré-visualização com válidas e rejeitadas.
7. Exigir confirmação POST protegida por CSRF.
8. Criar importacoes_excel e persistir linhas com savepoint individual.
9. Guardar em linhas_importacao o estado, erro e dados normalizados.
10. Apresentar relatório final com contagens e mensagens de rejeição.

Extra implementado. Os ficheiros em exemplos/ fornecem dois modelos simplificados já preenchidos. O parser aceita igualmente a folha Lista de Ativos do modelo CNCS fornecido. O segundo ficheiro original contém cabeçalhos, mas não dados reais, pelo que o parser informa corretamente que não existem linhas a importar.

10. Segurança e integridade

Medida	Implementação	Classificação
Passwords	PBKDF2-SHA256 através de make_password/check_password.	Obrigatória
SQL injection	Placeholders %s e parâmetros separados.	Obrigatória
CSRF	Middleware e tokens em todos os formulários POST.	Obrigatória
Autorização	Decorators por perfil e verificação de acesso ao cliente.	Obrigatória
Sessões	Cookies assinados, HttpOnly e SameSite=Lax.	Obrigatória
Uploads	Extensão, MIME, tamanho, nome aleatório e caminho privado.	Obrigatória
Ficheiros	SHA-256 e download apenas por view autorizada.	Recomendada
CORS	Não aplicável: a interface e backend são a mesma aplicação Django.	-
Segredos	Variáveis .env; apenas .env.example é entregue.	Obrigatória
Erros	Mensagem genérica ao utilizador e detalhe no log do servidor.	Recomendada
Auditoria	logs_atividade sem passwords/conteúdo de documentos.	Recomendada
Backups	Procedimento operacional a definir no deploy.	Evolução

10.1. Dados pessoais e sensíveis

Nomes, emails, telefones, NIF, endereços IP, inventário, incidentes, relatórios e pentests podem ser pessoais, confidenciais ou operacionalmente sensíveis. A aplicação aplica minimização de exposição: não serve a pasta privada como estática, restringe por cliente e perfil, não imprime credenciais e não inclui o ficheiro .env na entrega.

11. Testes e validação

11.1. Verificação executada

O script `scripts/verificar_projeto.py` foi executado contra PostgreSQL 17 e confirmou os elementos seguintes:

- Ligação à base `ciberbox_bd` e presença das 17 tabelas do domínio.
- Autenticação correta e rejeição de password errada.
- Execução das cinco consultas do dashboard e valores não vazios.
- CREATE, SELECT, UPDATE e DELETE temporários de um cliente, revertidos no fim.
- Leitura dos dois modelos Excel simplificados.
- Login HTTP e resposta das rotas de clientes, utilizadores, ativos, incidentes, avaliações, documentos, pedidos, importações e logs.
- Isolamento: Cliente 1 recebe 404 quando tenta abrir a organização 2.
- Autorização: o perfil Cliente é redirecionado quando tenta gerir utilizadores.

11.2. Resultados esperados do dashboard

Indicador	Resultado de demonstração
Conformidade	2 Conforme; 2 Em avaliação; 2 Com pendências.
Top incidentes	Zeta 5; Gamma 4; Beta 3; Alpha 2; Epsilon 2.
Documentos	Registos distribuídos entre janeiro e junho de 2026.
Perfis	1 Administrador; 1 Colaborador; 2 Clientes.
Pedidos	Estados preenchidos; média de 57 h em Resolvido e 48 h em Fechado.

11.3. Testes adicionais recomendados antes da defesa

- Executar a inicialização numa base de dados vazia num segundo computador.
- Importar o modelo de ativos duas vezes e demonstrar a rejeição dos inventários duplicados.
- Importar uma linha com IP/MAC inválido e confirmar que as restantes são guardadas.
- Tentar POST sem CSRF e confirmar rejeição.
- Tentar descarregar um documento de outro cliente.
- Demonstrar rollback de uma operação composta.
- Executar EXPLAIN ANALYZE nas cinco consultas com maior volume de dados.

12. Instalação e demonstração

12.1. Pré-requisitos

- Python 3.11 ou superior.
- PostgreSQL 15 ou superior.
- Acesso a psql ou pgAdmin para criar o utilizador e a base de dados.

12.2. Instalação

```
CREATE USER ciberbox_user WITH PASSWORD 'ciberbox_password';
CREATE DATABASE ciberbox_bd OWNER ciberbox_user ENCODING 'UTF8';
```

```
py -m venv .venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
python scripts/inicializar_bd.py --limpar
python manage.py check
```



```
python scripts/verificar_projeto.py
python manage.py runserver
```

12.3. Contas de demonstração

Perfil	Email	Password de demonstração
Administrador	admin@ciberbox.local	Demo2026!
Colaborador	colaborador@ciberbox.local	Demo2026!
Cliente Alpha	cliente1@ciberbox.local	Demo2026!
Cliente Beta	cliente2@ciberbox.local	Demo2026!

Nota de segurança. Estas contas existem apenas em dados de demonstração. Num ambiente real devem ser substituídas, DJANGO_SECRET_KEY deve ser aleatória, DEBUG deve estar desligado e os cookies devem ser Secure.

12.4. Guião breve de demonstração

1. Entrar como Administrador e apresentar os cinco blocos do dashboard.
2. Abrir a lista e detalhe de um cliente, relacionando os dados com as tabelas.
3. Criar ou editar um ativo e explicar a query parametrizada.
4. Abrir um pedido, responder e alterar estado, mostrando mensagens e histórico.
5. Importar o modelo_importacao_ativos.xlsx, rever a pré-visualização e confirmar.
6. Entrar como Cliente Alpha e demonstrar que só vê a organização associada.
7. Abrir o diagrama e explicar uma entidade independente, uma dependente e a N:M.
8. Explicar uma das consultas com JOIN, GROUP BY e agregação.

13. Limitações e trabalho futuro

- O projeto é um demonstrador académico e não foi submetido a teste de penetração independente.
- A aplicação é executada localmente; o deploy não faz parte desta entrega de Bases de Dados.
- Não existe recuperação de password por email nem MFA.
- As permissões são baseadas em três perfis fixos, não numa matriz granular configurável.
- Os dados de demonstração de documentos são metadados; uploads reais devem ser efetuados pela interface.
- A importação trata os modelos fornecidos e variantes de cabeçalhos conhecidas, mas não substitui um processo ETL universal.
- Para grandes volumes devem ser acrescentadas paginação, tarefas assíncronas e monitorização.
- A classificação NIS2 é uma informação funcional da aplicação e não constitui parecer jurídico.

14. Conclusão

O CiberBoxSecurBD satisfaz o núcleo formal da UC: apresenta os três níveis de modelação, SQL de criação e inicialização, aplicação Django e dashboard da Ficha 9. A persistência é realizada por SQL direto e parametrizado, sem ORM para o domínio. O modelo encontra-se normalizado até 3FN, usa constraints e índices orientados às regras e consultas e preserva histórico onde é necessário.

A funcionalidade extra de Excel demonstra a integração entre estruturas tabulares externas, validação e transações relacionais. O conjunto de dados de demonstração e o verificador automático tornam as decisões observáveis e repetíveis, o que facilita a entrega e a defesa técnica.

Referências

1. Enunciado do Trabalho Prático de Bases de Dados, ano letivo 2025/2026, pp. 1-6.
2. Ficha 2 - modelo conceptual e IDEF1X.
3. Ficha 3 - cardinalidades, dependências e especialização.
4. Ficha 5 - modelo lógico, modelo físico e normalização.
5. Ficha 6 - instruções SQL.
6. Ficha 7 - criação de projeto Django e basededados.py, pp. 1-3.
7. Ficha 8 - aplicação Django ao projeto prático, p. 1.
8. Ficha 9 - dashboard com cinco consultas SELECT, p. 1.
9. Requisitos Projeto Integrado III, pp. 1-2.
10. Exemplo inventario p_ CNCS.xlsx.
11. Exemplo Lista Activos _ incidentes.xlsx.
12. Documentação oficial PostgreSQL: <https://www.postgresql.org/docs/>
13. Documentação oficial Django: <https://docs.djangoproject.com/>

Anexo A - Estrutura da entrega

```
CiberBoxSecurBD/  
|-- ciberbox/      aplicação Django e SQL direto  
|-- ciberbox_bd/   configuração  
|-- sql/          scripts PostgreSQL  
|-- modelos/       diagramas e explicações  
|-- documentacao/  relatório e defesa  
|-- exemplos/      modelos Excel  
|-- scripts/       inicialização e verificação  
|-- .env.example  
|-- requirements.txt  
|-- manage.py  
`-- README.md
```

Anexo B – estrutura do que está feito

Elemento	Pontos feitos
clientes	PK, NIF UNIQUE/CHECK, relações 1:N e política ON DELETE.
utilizadores_clientes	Razão da N:M e PK composta.
avaliacoes_risco	Histórico e query da avaliação mais recente.
ativos/incidentes	Unicidade por cliente e ligação opcional à importação.
documentos	Metadados na BD e conteúdo em armazenamento privado.
pedidos/histórico	Estado atual versus histórico e cálculo do tempo.
normalização	Exemplos concretos de 1FN, 2FN e 3FN.
SQL direto	Placeholders, cursor, commit/rollback e ausência de ORM.
dashboard	JOIN, LEFT JOIN, GROUP BY, COUNT, AVG, FILTER e DATE_TRUNC.
Excel	Mapeamento, validação, preview, savepoints e relatório por linha.